

# Guía de `rtm32.asm`

Arquitectura del Computador

v1.1.0



## 1. Instalación de *rtm32.asm*.

En esta guía se detalla el proceso de instalación y uso básico del assembler *rtm32.asm* para la máquina RTM32.

El assembler se provee como un archivo estáticamente linkeado, por lo que no requiere instalar dependencias adicionales.

Se proveen ejecutables para las siguientes plataformas:

- + *aarch64-linux-musl*
- + *aarch64-macos*
- + *x86\_64-linux-musl*
- + *x86\_64-macos*
- + *x86\_64-windows-gnu*

Únicamente debes descargar el ejecutable que corresponda a tu plataforma. Si no sabés cuál elegir y estás en Linux, usá *x86\_64-linux-musl-rtm32.asm*.

Una vez descargado, quitá la plataforma del nombre del archivo, renombrándolo a *rtm32.asm*. De ser necesario, darle además permisos de ejecución.

```
$ mv x86_64-linux-musl-rtm32.asm rtm32.asm
$ chmod +x rtm32.asm
```

Con esto, ya podés usar el assembler. Podés verificarlo con el siguiente comando:

```
$ ./rtm32.asm --version
./rtm32.asm version 1.2.0
```

### 1.1. Recomendado: Instalarlo en PATH.

Para no tener que escribir *./rtm32.asm* o *~/Descargas/rtm32.asm*, se recomienda colocar al assembler en una carpeta que esté en tu *PATH* de usuario. No es estrictamente necesario pero se recomienda realizar este proceso.

Esto aplica para todas las plataformas soportadas. Sin embargo, qué carpetas forman parte del *PATH* y cómo modificarlo depende de tu sistema operativo, distribución y configuración. Es imposible detallarlo en solo este documento.

Podés consultar las carpetas actualmente incluidas en tu *PATH* con:

```
$ echo $PATH
```

Dónde instalar el ejecutable y cómo modificar el *PATH* queda a tu criterio o al de quien administre el sistema.

Se describen a continuación algunas rutas donde se puede colocar el archivo. Ojo que si alguna de estas carpetas *no* existe o *no* está en *PATH*, mover el archivo ahí no va a hacer nada!

### 1.1.1. Instalarlo para un solo usuario (ideal)

- + ~/.local/bin en sistemas Unix.
- + %LOCALAPPDATA%\Programs en Windows.

### 1.1.2. Instalarlo para todos los usuarios (aceptable, requiere privilegios de administrador)

- + /usr/local/bin en sistemas Unix

### 1.1.3. Instalarlo en una ruta del sistema

No se recomienda hacer esto, pero de ser muy complejo modificar tu PATH, estas “van a funcionar”:

- + /usr/bin en sistemas Unix.
- + %WINDIR%\System32 en Windows.

---

Para verificar que sea accesible globalmente, ahora deberías poder correr:

```
$ rtm32.asm --version
rtm32.asm version 1.2.0
```

## 2. Uso de rtm32.asm

Habiendo instalado el assembler, ya es posible generar programas. Para correr el assembler, se le debe indicar por la línea de comandos un archivo de entrada y un archivo de salida. Esto se hace de la siguiente manera:

```
$ rtm32.asm mi_programa_en_assembly.rtm -o mi_archivo_ejecutable.bin
```

Esto leerá el archivo de texto `mi_programa_en_assembly.rtm` y generará el ejecutable `mi_archivo_ejecutable.bin`: Podrás correr esto con el emulador de RTM32. Para que un archivo sea ejecutable, además de ser sintácticamente válido se deben poder resolver todos los símbolos. Es decir, si escribiste `j tag1234`, entonces en algún otro lugar debe estar definida la etiqueta `tag1234`.

Alternativamente, se puede generar un *archivo de objeto*. Esto se hace agregando la flag `-c` a la invocación:

```
$ rtm32.asm -c mi_archivo_en_assembly.rtm -o mi_objeto.o
```

`mi_objeto.o` **no** es un ejecutable, sino que es más parecido a una librería. Las etiquetas definidas en `mi_archivo_en_assembly.rtm` se podrán usar en cualquier archivo que luego esté linkeado con este. Por ejemplo, si `mi_programa_en_assembly` quiere usar estas etiquetas, lo linkeamos de la siguiente manera:

```
$ rtm32.asm -c mi_archivo_en_assembly.rtm -o mi_objeto.o
$ rtm32.asm mi_programa_en_assembly.rtm -o mi_archivo_ejecutable.bin -l:mi_objeto.o
```

La flag `-l:<archivo>` se puede usar cualquier cantidad de veces e indica que se debe linkear con este objeto adicional.

## 2.1. Programa de ejemplo: Hola mundo

Se comparte aquí un programa comentado que imprime “Hola, mundo!” por la consola del emulador. Vamos a usarlo de ejemplo para ver cómo usar los programas generados por `rtm32.asm` con el emulador `RTM32`.

```
.section ".data"
    mensaje: .asciiz "Hola, mundo!"

.section ".text"
    // Elegimos un valor para inicializar el stack
    addi $sp, $zero, 0xFFC

    // Para llamar a `imprimir_asciiz` debemos cargar en $a0 un puntero
    // al mensaje. Un puntero es de 32 bits, por lo cual es imposible hacerlo
    // con tan solo una instrucción.

    // Hay dos formas de cargar un puntero. La primera es usar las relocalaciones
    // .lo16 y .hi16 para cargarlo en dos pasos. La segunda es bakear el puntero
    // en el segmento de datos con `mensaje_ptr: .ptr mensaje` y cargar el puntero
    // directamente con `lw $rt, mensaje_ptr($zero)`.

    // Usamos la primer forma:
    orh $a0, $zero, .hi16 mensaje
    ori $a0, $a0, .lo16 mensaje

    jal imprimir_asciiz

    // Debemos detener la ejecución del programa aquí. De otro modo, el
    // emulador volverá a avanzar hacia imprimir_asciiz. No tenemos una
    // instrucción HALT, pero si estamos corriendo la máquina con el debugger,
    // podemos obtener el mismo ejemplo con una excepción:
    trap 1
    // ~ Esta excepción no está definida, por lo cual el debugger detendrá la
    // ejecución por nosotros con "Core Exception 6."

    // Hagamos una función "imprimir_asciiz"
    // Tomará en $a0 un puntero a la string que quiere imprimir
    imprimir_asciiz:
        // Los registros $sX son callee-saved. Es decir, si los usamos dentro
        // de esta función, debemos restaurarlos a su valor original antes de
        // salir. Esto se suele hacer empujándolos y quitándolos del stack.
        addi $sp, $sp, -12
        sw $s0, 0($sp)    // guarda puntero al próximo caracter
        sw $s1, 4($sp)    // guarda el caracter actual
        sw $s2, 8($sp)    // guarda la dirección de la consola
        // No hace falta empujar $ra porque no llamamos a ninguna derivada de JAL

        // Al principio no leímos ningún caracter. Establecemos $s0 al principio
        // de la string, es decir, en $a0
        addi $s0, $a0, 0

        // Truco para guardar la dirección de la consola en una instrucción:
        addi $s2, $zero, -0x100
```

```

    // Procesamos los caracteres uno por uno
imprimir_asciiz.bucle:
    lbu $s1, 0($s0)

    // Una string .asciiz termina cuando lleguemos a un byte cuyo valor es 0
    beq $s1, $zero, imprimir_asciiz.salir

    // Si llegamos acá, $s1 != 0, lo imprimimos:
    sb $s1, 0($s2)

    // Avanzamos el puntero, y la próxima vuelta procesamos el siguiente caracter:
    addi $s0, $s0, 1
    j imprimir_asciiz.bucle

imprimir_asciiz.salir:
    // Imprimamos tambien un salto de línea. Nosotros estamos acostumbrados a
    // escribirlos con '\n'. Esto se conoce como "LF." En RTM32, sin embargo,
    // se usa "CRLF." Es decir, debemos imprimir dos caracteres, '\r' y '\n',
    // donde '\r' mueve el cursor hacia el borde izquierdo de la consola, y '\n'
    // únicamente lo desplaza una celda hacia abajo sin restablecer la posición
    // horizontal.

    addi $s1, $zero, '\r'
    sb $s1, 0($s2)

    addi $s1, $zero, '\n'
    sb $s1, 0($s2)

    // Ahora sí, salimos de la función:
    // Empujamos varias cosas al stack, debemos restaurarlas
    lw $s0, 0($sp)
    lw $s1, 4($sp)
    lw $s2, 8($sp)
    addi $sp, $sp, 12

    // Ahora si, salimos de la función
    jr $ra, 0

```

Lo guardamos en el archivo `hola_mundo.rtm`. Ensamblamos:

```
$ rtm32.asm hola_mundo.rtm -o hola_mundo.bin
```

Asumiendo que `rtm32 -d telnet` ya está corriendo, nos conectamos por telnet y cargamos el programa:

```

$ telnet -4 localhost 4444
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
RTM32-0.1.1
RTM32> load hola_mundo.bin
Successfully burst loaded 116 aligned bytes into base address 0x00000000

```

Podemos probar correrlo. Como se indicó en los comentarios del código, el programa tiene una forma “sucía” de terminar su ejecución, por lo cual deberíamos esperar ver el error “Core Exception 6”

```
RTM32> c
Continuing execution...

[Execution Fault: Core Exception 6 thrown at PC 0x00000014]
Core execution stopped. Current PC: 0x00000014
RTM32> r
=== General Purpose Registers ===
R[ 0]: 0x00000000  R[ 1]: 0x00000010  R[ 2]: 0x00000000  R[ 3]: 0x00000000
R[ 4]: 0x00000000  R[ 5]: 0x00000000  R[ 6]: 0x00000000  R[ 7]: 0x00000000
R[ 8]: 0x00000064  R[ 9]: 0x00000000  R[10]: 0x00000000  R[11]: 0x00000000
R[12]: 0x00000000  R[13]: 0x00000000  R[14]: 0x00000000  R[15]: 0x00000000
R[16]: 0x00000000  R[17]: 0x00000000  R[18]: 0x00000000  R[19]: 0x00000000
R[20]: 0x00000000  R[21]: 0x00000000  R[22]: 0x00000000  R[23]: 0x00000000
R[24]: 0x00000000  R[25]: 0x00000000  R[26]: 0x00000000  R[27]: 0x00000000
R[28]: 0x00000000  R[29]: 0x00000000  R[30]: 0x0000FFFC  R[31]: 0x00000000

=== Control & Special Registers ===
PC      : 0x00000014  CAUSE  : 0x00000006  EPC      : 0x00000000
BADVADR : 0x00000000  VBR    : 0x00000002

Execution State:
Mode: KERNEL | Flags: [-ZC--]

Last Memory Operation:
Address: 0x00000010 | Size: 0x00000004 | Type: FETCH
```

Y efectivamente, si revisamos la consola de picocom, vemos la frase: Hola, mundo!

### 3. Assembly de `rtm32.asm`.

Assembly es una familia de lenguajes de programación de bajo nivel, donde la mayoría de líneas de código corresponden uno a uno contra las instrucciones de máquina que se van a ejecutar. Aquí se detallan las especificaciones de la sintaxis aceptada por el assembler `rtm32.asm` para el procesador STX4.

Conceptualmente, `rtm32.asm` funciona realizando un gran pattern matching sobre las *sentencias* un archivo de texto entero, codificando cada una como código de RTM32. Las sentencias tienen un formato predefinido dependiendo de su operación. En general cada línea corresponde a una sentencia distinta. De ser necesario, se pueden ingresar dos sentencias en la misma línea insertando un punto y coma (;) entre ellas, y se puede separar una sentencia en múltiples líneas estableciendo un \ al final.

Las sentencias se clasifican en dos grandes grupos: Directivas, e instrucciones.

Adicionalmente, se soporta un conjunto de herramientas de preprocesador, detalladas más adelante, aunque vale remarcar que estos *no son* sentencias, sino que herramientas para armarlas.

#### 3.1. Tipos de palabras.

`rtm32.asm` no opera carácter por carácter, sino que las agrupa en “*tokens*” de distinto tipo. Las instrucciones y directivas como una secuencia de tokens. Se reconocen los siguientes tipos de tokens:

- ✦ Los siguientes caracteres sueltos, cada uno siendo su propio tipo: ., ,, ;, (, ), \$, -, +, ~.
- ✦ Un *número*, que se puede escribir de cuatro formas:
  - Directamente en base 10, como 1206.
  - En base 2, como 0b10010110110.
  - En base 8, como 0o2266.
  - En base 16, como 0x4B6 o 0x4b6.
- ✦ Un *identificador* es una secuencia de caracteres. El primero de estos caracteres está entre a y z o A y Z. Cada uno de los demás caracteres está entre a–z, A–Z, 0–9, o es un ., \_, o '.
- ✦ Un *carácter* se escribe entre comillas simples (') y se puede escapar. Representan siempre un byte.
- ✦ Una *string* es una secuencia de caracteres. En lugar de encerrar cada carácter entre 's, únicamente se encierra a la string entera entre "s. Cero o más caracteres dentro de la string pueden estar escapados, detallado más adelante.
- ✦ Un *byte* puede ser un carácter o un número literal entre 0 y 255.
- ✦ Un *registro* se escribe con un \$ seguido de su número, o, su nombre. Por ejemplo \$0 y \$zero representan el mismo registro, y lo mismo con \$1 y \$ra.
- ✦ Un *registro especial* se escribe con un \$ seguido de su número, o, su nombre. Por ejemplo \$0 y \$psw representan el mismo registro especial.
- ✦ Una *etiqueta* es un identificador seguido de un :. Se usa para ponerle un nombre a una sentencia y poder referirse a ella en otro lugar.

- + Un *inmediato* puede ser un número, un byte, o un puntero a una etiqueta con un desplazamiento.
  - Según dónde se solicite el inmediato, puede tomar un tamaño distinto (ELIMM, VLIMM, LLIMM, LIMM, ...)
- + Hay dos formas de escribir *comentarios*: Los comentarios de línea comienzan con `//` y terminan en el primer salto de línea que encuentren (no inclusive,) y los bloques de comentario comienzan en un `/*` y continúan hasta encontrar su terminador, `*/`.

Los registros generales y los registros especiales utilizan la misma sintaxis. Cuando un número o nombre puede corresponder a ambos, el tipo de registro esperado depende de la instrucción que está siendo analizada.

Se reserva el nombre de la etiqueta `“heap_base.”` No se puede declarar una etiqueta con ese nombre pero si se puede utilizar. Su valor está alineado a 4 bytes y apunta al final de las secciones de código y datos. Es decir, al comienzo del *heap*.

“*Escapar*” un caracter significa reemplazarlo por otro byte que no se puede fácilmente escribir en el teclado, o que generaría un error léxico al insertarlo. Un escape arranca con el caracter `\`, y se reconocen los siguientes escapes:

- + `\0` inserta el byte `0x00` (NUL).
- + `\n` inserta el byte `0x0A` (LF).
- + `\r` inserta el byte `0x0D` (CR).
- + `\t` inserta el byte `0x09` (Tab).
- + `\'` inserta el byte `0x27` (').
- + `\"` inserta el byte `0x22` (").
- + `\xXY` requiere de dos caracteres adicionales, e insertará el byte literal `0xXY`.

### 3.2. Directivas.

Las **directivas** se utilizan para escribir datos literales, o cambiar el estado del assembler. Cada directiva válida comienza con un `.` seguida por un identificador.

Entre el nombre, entre cada uno de los términos, al principio, o al final de cada directiva puede haber espacio horizontal (espacios o tabulaciones.) De necesitarse espacio vertical (un salto de línea) se debe finalizar la línea con `\`.

Se reconocen las siguientes directivas. Referirse al apartado anterior para conocer cada tipo de palabra:

- + `.section <identificador | string>` toma el nombre de una sección y le indica al assembler que a partir de ahora escriba los datos ahí. No se codifica en el programa.
  - Por ejemplo, podés escribir `.section ".text"` para comenzar a escribir tu código.
  - De mismo modo, podés escribir `.section ".data"` para comenzar a escribir datos de lectura y escritura.
- + `.ascii <string...>` codifica una string como una secuencia de bytes, terminada por un byte NUL (`0x00`.) Se codifica alineado a 1 byte.

- `.ascii "HOLA"` se codifica como `48 4f 4c 41 00`.
- + `.nascii <string...>` codifica una string como una palabra representando su longitud seguida por una secuencia de bytes representando sus datos. Se codifica alineado a 4 bytes.
  - `.nascii "HOLA"` se codifica como `00 00 00 04 48 4f 4c 41`.
- + `.byte <bytes...>` inserta una secuencia de bytes literales. Se codifica alineado a 1 byte.
  - `.byte 1 2 'A' 3` se codifica como `01 02 48 03`.
- + `.short <números...>` inserta una secuencia de números de 16 bits literales, rellenos con ceros de ser necesario. Se codifica alineado a 2 bytes.
- + `.word <números...>` inserta una secuencia de números de 32 bits literales, rellenos con ceros de ser necesario. Se codifica alineado a 4 bytes.
  - `.word 1 2 3` se codifica como `00 00 00 01 00 00 00 02 00 00 00 03`.
  - `.word 1 'A' 3` no es sintaxis válida.
- + `.pad <números...>[(<alineación>)]` recibe una secuencia de números, computa la suma, y luego inserta esa misma cantidad de ceros. Pegado al último número puede ir otro valor encerrado en paréntesis, indicando que el cursor final debe estar alineado a un múltiplo de esa cantidad de bytes. En este caso, rellena con más ceros hasta llegar al próximo múltiplo. De no escribir un valor para la alineación se toma 1 como valor implícito.
  - `.pad 10` inserta 10 ceros en esta sección.
  - `.pad 1 2 3` inserta 6 ceros en esta sección, porque  $1 + 2 + 3 = 6$ .
  - Al principio de la sección, `.pad 1 2 3(4)` termina insertando 8 ceros en lugar de 6, porque debe rellenar hasta el próximo múltiplo de 4.
  - Al principio de la sección, `.pad 2 2(4)` inserta 4 bytes, al ya estar alineado a un múltiplo de 4, no necesita agregar nada más.
  - `.pad 1 2(4) 3` no es sintaxis válida.
- + `.ptr <identificador>` escribe una palabra con la dirección de memoria de la sentencia que tiene esta etiqueta. Se codifica alineado a 4 bytes.
- + `.lo16 <identificador>` escribe los 16 bits más bajos de la dirección de memoria a la sentencia que tiene esta etiqueta. Se codifica alineado **al final** de 4 bytes
- + `.hi16 <identificador>` escribe los 16 bits más altos de la dirección de memoria a la sentencia que tiene esta etiqueta. Se codifica alineado **al final** de 4 bytes
  - Esta codificación extraña permite escribir instrucciones como `orh $a0, $a0`, `.hi16 msg` y `ori $a0, $a0`, `.lo16 msg`, que de otro modo no serían posibles.

### 3.3. Instrucciones.

Todas las **instrucciones** son de la forma `<nombre> <argumentos...>` y la aridad de cada instrucción está entre 0 y 4. Los argumentos se separan con comas. En particular, los formatos de sintaxis **son distintos** a la especificación de operandos en el manual de RTM32. El funcionamiento de cada instrucción es idéntico al que se describe en el manual, porque el assembler jamás puede modificarlo.

La mayor diferencia sintáctica es que todas las instrucciones tienen su registro de destino como *primer argumento*, mientras que en el manual y la codificación suele ser el último.

Entre el nombre, entre cada uno de los argumentos, al principio, o al final de cada instrucción puede haber espacio horizontal (espacios o tabulaciones.) De necesitarse espacio vertical (un salto de línea) se debe finalizar la línea con \.

Todo campo `imm` representa un inmediato. Puede ser escrito como un número en cualquier base, o como el nombre de una etiqueta. Al escribir el nombre de una etiqueta es posible establecer un desplazamiento. Por ejemplo, `lw $t0, arr+4($zero)` cargará la palabra 4 bytes después de `arr`.

En las siguientes tablas se describen los formatos de cada instrucción.

### 3.3.1. Tipo R.

| Instr               | Sintaxis                             |
|---------------------|--------------------------------------|
| <code>sll</code>    | <code>sll \$rd, \$rt, aux</code>     |
| <code>rlc</code>    | <code>rlc \$rd, \$rt, aux</code>     |
| <code>srl</code>    | <code>srl \$rd, \$rt, aux</code>     |
| <code>sra</code>    | <code>sra \$rd, \$rt, aux</code>     |
| <code>sllr</code>   | <code>sllr \$rd, \$rt, \$rs</code>   |
| <code>rlcr</code>   | <code>rlcr \$rd, \$rt, \$rs</code>   |
| <code>srlr</code>   | <code>srlr \$rd, \$rt, \$rs</code>   |
| <code>srar</code>   | <code>srar \$rd, \$rt, \$rs</code>   |
| <code>and</code>    | <code>and \$rd, \$rs, \$rt</code>    |
| <code>or</code>     | <code>or \$rd, \$rs, \$rt</code>     |
| <code>nor</code>    | <code>nor \$rd, \$rs, \$rt</code>    |
| <code>xor</code>    | <code>xor \$rd, \$rs, \$rt</code>    |
| <code>add</code>    | <code>add \$rd, \$rs, \$rt</code>    |
| <code>sub</code>    | <code>sub \$rd, \$rs, \$rt</code>    |
| <code>slt</code>    | <code>slt \$rd, \$rs, \$rt</code>    |
| <code>sltu</code>   | <code>sltu \$rd, \$rs, \$rt</code>   |
| <code>lhx</code>    | <code>lhx \$rd, \$rs, \$rt</code>    |
| <code>lhux</code>   | <code>lhux \$rd, \$rs, \$rt</code>   |
| <code>lbx</code>    | <code>lbx \$rd, \$rs, \$rt</code>    |
| <code>lbux</code>   | <code>lbux \$rd, \$rs, \$rt</code>   |
| <code>lwx</code>    | <code>lwx \$rd, \$rs, \$rt</code>    |
| <code>swx</code>    | <code>swx \$rd, \$rs, \$rt</code>    |
| <code>shx</code>    | <code>shx \$rd, \$rs, \$rt</code>    |
| <code>sbx</code>    | <code>sbx \$rd, \$rs, \$rt</code>    |
| <code>mul</code>    | <code>mul \$rd, \$rs, \$rt</code>    |
| <code>mulh</code>   | <code>mulh \$rd, \$rs, \$rt</code>   |
| <code>mulhu</code>  | <code>mulhu \$rd, \$rs, \$rt</code>  |
| <code>mulhsu</code> | <code>mulhsu \$rd, \$rs, \$rt</code> |
| <code>div</code>    | <code>div \$rd, \$rs, \$rt</code>    |
| <code>divu</code>   | <code>divu \$rd, \$rs, \$rt</code>   |
| <code>rest</code>   | <code>rest \$rd, \$rs, \$rt</code>   |
| <code>restu</code>  | <code>restu \$rd, \$rs, \$rt</code>  |
| <code>cfs</code>    | <code>cfs \$rs, aux</code>           |
| <code>cts</code>    | <code>cts \$rs, aux</code>           |
| <code>trap</code>   | <code>trap aux</code>                |

| Instr | Sintaxis |
|-------|----------|
| rft   | rft      |

### 3.3.2. Tipo I.

| Instr | Sintaxis                 |
|-------|--------------------------|
| jr    | jr \$rs, llimm           |
| jalr  | jalr \$rs, llimm         |
| jalrx | jalrx \$rs, \$lrX, llimm |
| addi  | addi \$rt, \$rs, simm    |
| andi  | andi \$rt, \$rs, simm    |
| lci   | lci \$rt, \$rs, simm     |
| ani   | ani \$rt, \$rs, simm     |
| anh   | anh \$rt, \$rs, simm     |
| ori   | ori \$rt, \$rs, simm     |
| orh   | orh \$rt, \$rs, simm     |
| xori  | xori \$rt, \$rs, simm    |
| xorh  | xorh \$rt, \$rs, simm    |
| lw    | lw \$rt, imm(\$rs)       |
| sw    | sw \$rt, imm(\$rs)       |
| sh    | sh \$rt, imm(\$rs)       |
| sb    | sb \$rt, imm(\$rs)       |
| lh    | lh \$rt, imm(\$rs)       |
| lhu   | lhu \$rt, imm(\$rs)      |
| lb    | lb \$rt, imm(\$rs)       |
| lbu   | lbu \$rt, imm(\$rs)      |
| beq   | beq \$rt, \$rs, imm      |
| bne   | bne \$rt, \$rs, imm      |
| blt   | blt \$rt, \$rs, imm      |
| bge   | bge \$rt, \$rs, imm      |
| bltu  | bltu \$rt, \$rs, imm     |
| bgeu  | bgeu \$rt, \$rs, imm     |
| slti  | slti \$rt, \$rs, imm     |
| sltiu | sltiu \$rt, \$rs, imm    |

### 3.3.3. Tipo J.

| Instr | Sintaxis          |
|-------|-------------------|
| j     | j elimm           |
| jal   | jal elimm         |
| jalx  | jalx \$lrX, vlimm |

## 3.4. Preprocesador.

El *preprocesador* de `rtm32.asm` es un procesador de macros utilizado automáticamente por el ensamblador para transformar tu código fuente antes de ensamblarlo. Se llama “preprocesador de macros” porque te permite definir *macros*, que son abreviaciones breves

para construcciones más grandes. Ninguna palabra del preprocesador es una instrucción de código, y El preprocesador de *rtm32.asm* funciona en base a directivas de preprocesador y expansiones de macros.

Todas las directivas de preprocesador comienzan con un `%`. Se reconocen las siguientes directivas de preprocesador:

- + `%filename` inserta una string conteniendo el nombre del archivo donde se encontró esta directiva.
- + `%line` inserta una string conteniendo el número de línea dentro del archivo donde se encontró esta directiva.
- + `%ifdef <nombre>` e `%ifndef <nombre>` deben estar finalizadas por un `%endif` cada una, y pueden contener un `%else` dentro de ellas. Condicionalmente, dependiendo de si está definido un macro llamado `<nombre>`, dejarán el cuerpo correspondiente y eliminarán el otro.
- + `%include <archivo>` copia y pega el contenido de `<archivo>` sobre esta directiva sin tokenizar.
- + `%embed <archivo>` copia y pega el contenido de `<archivo>` sobre esta directiva como bytes.
- + `%define <nombre> <valor>` crea un nuevo macro de aridad 0 cuyo cuerpo es `<valor>`, hasta el salto de línea (sin incluir.)
- + `%macro <nombre>(<argumentos...>)` comienza a crear un nuevo macro llamado `<nombre>`. `<argumentos...>` es una lista separada por comas de nombres de macros temporales que podrás usar como argumentos, y se puede dejar vacío para tener un macro de aridad 0. Todo lo que escribas luego de esta línea hasta `%endmacro` (no inclusive) formará parte del cuerpo del macro.

Además, se proveen dos macros propios del assembler, que no podés redefinir:

- + `@sizeof(<data type>)` toma una directiva de datos y devuelve un número con su tamaño en bytes.
- + `@alignOf(<data type>)` toma una directiva de datos y devuelve un número con su alineación en bytes.
- + `@stringify(...)` toma tokens en crudo y devuelve una string que las representa.

Para consultar cómo termina el archivo únicamente tras haber ejecutado el preprocesador, ejecutá el comando:

```
rtm32.asm -E <entrada.rtm> -o <entrada_preprocesada.rtm>
```